

New York University Tandon School of Engineering
Computer Science and Engineering

CS-GY 6923: Written Homework 1.
Due Thursday, Oct. 02, 2025, 11:59pm.

Discussion with other students is allowed for this problem set, but solutions must be written-up individually.

For this first problem set, 10% extra credit will be given if solutions are typewritten (using LaTeX, Markdown, or another mathematical formatting program). MSWord with Equation Editor does not count.

Problem 1: Minimizing a Weighted Loss Function (15pts)

Consider the standard multivariate linear model of the form:

$$f_{\beta}(\mathbf{x}) = \langle \beta, \mathbf{x} \rangle.$$

In class, we saw how to optimize this model under the sum-of-squares loss, which for a training data set $(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)$ is equal to:

$$L_{SS}(\beta) = \sum_{i=1}^n (\langle \beta, \mathbf{x}_i \rangle - y_i)^2.$$

Sometimes, we are interested instead in using a *weighted* ℓ_2 loss that weights different training points differently. Specifically, suppose we are given a list of positive weights $w_1, \dots, w_n > 0$. We might want to minimize the *weighted sum-of-squares* loss:

$$L_{WSS}(\beta) = \sum_{i=1}^n w_i \cdot (\langle \beta, \mathbf{x}_i \rangle - y_i)^2.$$

Weighted loss functions are often used when some of your data is considered “more reliable” (e.g., because it was collected with a more accurate instrument). You might set the weights for the more reliable examples higher than those of less reliable examples.

- Derive an expression for the gradient of $L_{WSS}(\beta)$. Note that $w_1, \dots, w_n$ are fixed constants given to us in advance. They are not parameters of the model.
- Write down a closed form expression for $\beta^* = \arg \min_{\beta} L_{WSS}(\beta)$. I.e., for the parameters that minimize the weighted loss. Your expression should include the given weights $w_1, \dots, w_n$ in some way.
- Asymmetric Cost Minimization.** Let the residue $r_i = \langle \beta, \mathbf{x}_i \rangle - y_i$. Suppose overestimation ($r_i > 0$) incurs cost $c_+ > 0$ per unit error and underestimation ($r_i < 0$) incurs cost $c_- > 0$ per unit error. Consider the asymmetric linear loss

$$L_{QL}(\beta) = \sum_{i=1}^n (c_+ \cdot \max\{0, r_i\} + c_- \cdot \max\{0, -r_i\})$$

Define the quantile loss for $\tau \in (0, 1)$ to be

$$\rho_{\tau}(r_i) = \begin{cases} \tau r & \text{if } r \geq 0 \\ (\tau - 1)r & \text{if } r < 0 \end{cases}$$

Show that $L_{QL}(\beta)$ loss can be rewritten in the form

$$L_{QL}(\beta) = (c_+ + c_-) \sum_{i=1}^n \rho_{\tau}(r_i)$$

for an appropriate choice of $\tau \in (0, 1)$.

Identify τ in terms of c_+ and c_- . Interpret the role of τ . What happens when $c_+ \gg c_-$? What about $c_- \gg c_+$? What about $c_+ = c_-$?

Problem 2: Machine Learning Does Averages (20pts)

Suppose we have data $y_1, \dots, y_n \in \mathbb{R}$ and we want to choose a single value $m \in \mathbb{R}$ which is “most representative” of our dataset. This is sometimes called the “central tendency” problem in statistics. A machine learning approach to this problem would measure how representative m is of the data using a loss function. As you will see, different choices of loss function lead to different measures of central tendency you have probably seen in the past!

- Consider the loss function $L(m) = \sum_{i=1}^n (y_i - m)^2$. Show that $L(m)$ is minimized by setting $m = \bar{y}$, where $\bar{y} = \frac{1}{n} \sum_{i=1}^n y_i$ is the **mean** of our data.
- Consider the loss function $L(m) = \max_i |y_i - m|$. What value of m minimizes this loss? **Hint:** Using derivatives will not help here – try just thinking about the minimization problem directly.
- Consider the loss function $L(m) = \sum_{i=1}^n |y_i - m|$. Prove that $L(m)$ is minimized by setting m to the **median** of the data. **Hint:** This question is harder than the previous two and takes some creativity! Again derivatives might not be helpful.
- One way of breaking the central tendency is by the means of quintile loss introduced in Problem 1. Consider the loss function $L(m) = \sum_{i=1}^n \rho_\tau(m - y_i)$ where the quantile loss ρ_τ is defined as in Problem 1. What value of m minimizes the loss? You do not need to derive the solution; instead, give a short explanation of the meaning of your answer, does this generalize part (c).

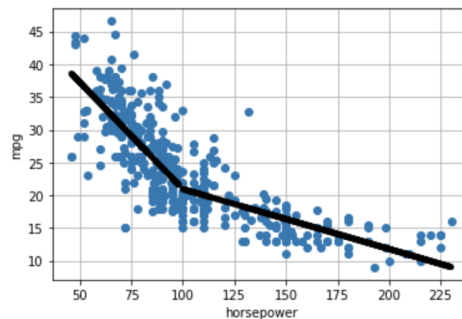
Problem 3: Piecewise Linear Regression via Feature Transformations (15pts)

Your goal is to fit a *piecewise* linear model to a single variate dataset of the form $(x_1, y_1), \dots, (x_n, y_n)$ where all values are scalars. We will only use two pieces. In other words, for some known value λ ,

$$f(x_i) = \begin{cases} a_1 + s_1 x_i & \text{for } x_i < \lambda \\ a_2 + s_2 x_i & \text{for } x_i \geq \lambda \end{cases}$$

with the additional **constraint** that $a_1 + s_1 \lambda = a_2 + s_2 \lambda$. This constraint ensures that our two linear models actually “meet” at $x = \lambda$, which means we get a continuous prediction function.

For example, when $\lambda = 100$, a piecewise linear fit for our MPG data might look like:



- Show that this model is equivalent to the following **unconstrained** model:

$$f(x_i) = \begin{cases} a_1 + s_1 x_i & \text{for } x_i < \lambda \\ a_1 + s_1 \lambda - s_2 \lambda + s_2 x_i & \text{for } x_i \geq \lambda \end{cases}$$

- Show how to fit an optimal f under the squared loss using an algorithm for multiple linear regression. In particular, your approach should:
 - Transform the input data to form a data matrix $\mathbf{X}$ with multiple columns.
 - Use a multiple regression algorithm to find the β which minimizes $\|\mathbf{y} - \mathbf{X}\beta\|_2^2$.

- Extract from the optimal β optimal values for a_1, s_1, s_2 .

You need to describe 1) a correct data transformation and 2) a correct mapping from β to a_1, s_1, s_2 . **Note that in our model λ is known. It is not a model parameter which needs to be optimized.**

- (c) Implement your algorithm in Python and apply it to the dataset from `demo_auto_mpg.ipynb`. Produce a piecewise linear fit for MPG as a function of Horsepower using the value $\lambda = 100$. Plot the result. You can attach a Jupyter notebook to your submission, or simply include the printed code and plot.

Problem 4: Thinking About Data Transformations (15pts)

Supposed you are trying to fit a multiple linear regression model for a given data set with the standard sum-of-squares loss function. You have already transformed your data by appending a column of all ones, which resulted in a final data matrix:

$$X = \begin{bmatrix} 1 & x_{1,1} & x_{1,2} & \dots & x_{1,d} \\ 1 & x_{2,1} & x_{2,2} & \dots & x_{2,d} \\ \vdots & \vdots & & & \vdots \\ 1 & x_{n,1} & x_{n,2} & \dots & x_{n,d} \end{bmatrix}$$

However, your model does not seem to be working well. It obtains poor loss in both training and test.

- (a) A friend suggests that you should try mean centering your data columns. In other words, for each i , compute the column mean $\bar{x}_i = \frac{1}{n} \sum_{j=1}^n x_{j,i}$ and subtract $\bar{x}_i$ from every entry in column i . Note that we won't mean center the first column, as doing so would set the 1s to 0s. You try this, but mean centering gives no improvement in the model loss at all.

Use a mathematical argument to explain why this is the case. **Hint:** It does not depend on the specific data set – mean centering will never help improve a multiple linear regression model!

- (b) Another friend suggests normalizing your data columns to have unit standard deviation. In other words for each i , compute the column standard deviation $\sigma_i = \sqrt{\frac{1}{n} \sum_{j=1}^n (x_{j,i} - \bar{x}_i)^2}$ and *divide* every column by σ_i . Again you try it, but normalizing gives no improvement in the model loss at all.

Use a mathematical argument to explain why this is the case.

- (c) Would your answers above change if you are using ℓ_1 loss or ℓ_∞ loss instead of sum-of-squares (ℓ_2) loss?